

PDC Assignment 2 - OpenMP Implementation and Performance

Xiang Li | Student ID: a1819316

1. Overview

This submission examines correctness and performance in a shared-memory OpenMP N-body solver. Part 2A addresses synchronization when OpenMP threads accumulate pairwise force contributions into a shared array using either a critical region or per-particle locks. Part 2B compares the Basic and Reduced algorithms and three Reduced scheduling strategies using runtime, speedup, efficiency and scalability.

2. Part 2A synchronization explanation

The reduced-force algorithm calculates each particle pair once and applies equal and opposite contributions to both particles. After the outer force loop is parallelised, different threads can update the same forces entry concurrently. Because these are read-modify-write operations, an overlap can lose a contribution, creating a race condition and nondeterministic results.

Both versions use a main parallel region with `default(none)`. `curr`, `forces`, simulation parameters and loop bounds are explicitly shared, while the timestep, particle index and simulation-time variables are private; the lock array is additionally shared in the lock version. `Reset_forces` uses `omp single`, while force calculation and body update use `omp for`. Their implicit barriers ensure initialization completes before force calculation and all forces are complete before body updates begin.

The critical implementation protects the four force updates from one pair interaction inside one `omp critical` region, preventing conflicting writes but serializing force accumulation through that region. The lock implementation creates one lock per particle and protects `forces[part]` and `forces[k]` separately. Only one particle lock is held at a time, allowing unrelated force entries to be updated concurrently.

3. Part 2A validation

Both implementations were compared with the supplied sequential shared-force reference using 1, 2, 4 and 8 threads. Tests used generated cases of 4 particles/2 timesteps and 8 particles/5 timesteps. Particle-state output was compared with diff after removing the elapsed-time line; all tested thread counts matched the reference.

4. Experimental environment and methodology

Part 2B ran in the supplied PDC Dev Container on Linux x86-64 with GCC 11.4.0 and OpenMP; 8 logical CPUs were visible. All implementations used 4000 particles, 20 timesteps, `delta_t = 0.01`, output frequency 20 and generated initial conditions. `NO_OUTPUT` disabled detailed output and `OMP_DYNAMIC` was `FALSE`. Thread counts were 1, 4, 8, 16 and 32, with five repetitions per configuration (100 raw timings). Mean runtime was reported. Speedup used $S_p = T_1/T_p$ with each implementation's own one-thread mean, and efficiency used $E_p = S_p/p$. The 16- and 32-thread cases oversubscribe the 8 logical CPUs.

5. Results

Table 1 gives mean runtime; Figures 1 and 2 show speedup and efficiency.

Table 1. Mean execution time (seconds)

Threads	Basic	Reduced Default	Forces Cyclic	All Cyclic
1	4.491	2.357	2.320	2.305
4	1.191	1.076	0.676	0.813
8	1.075	0.748	0.638	0.652
16	1.036	0.682	0.608	0.600
32	1.079	0.632	0.618	0.630

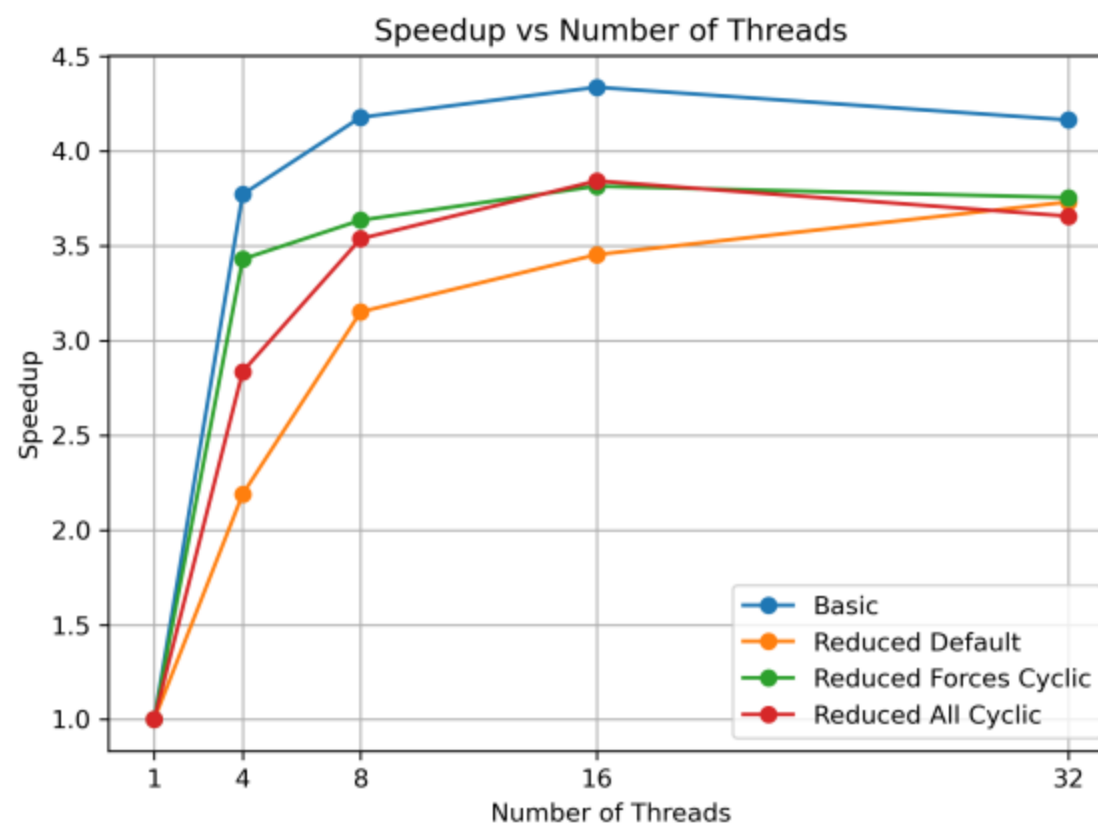


Figure 1. Speedup versus number of OpenMP threads.

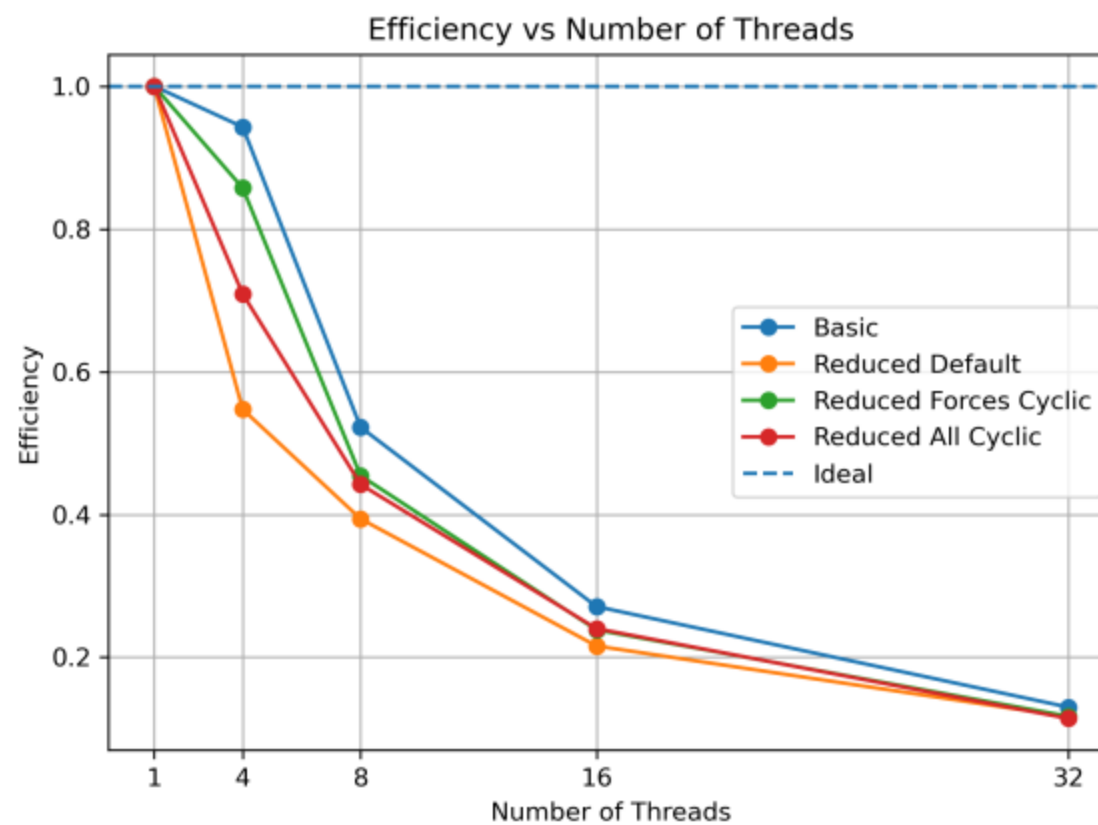


Figure 2. Efficiency versus number of OpenMP threads.

6. Performance analysis

The main algorithmic difference is visible at one thread: Basic took 4.491 s, while the Reduced versions took about 2.31-2.36 s. Basic calculates each ordered particle interaction, whereas Reduced calculates each pair once and reuses the equal-and-opposite contribution. Reduced therefore has lower absolute runtime. Basic still records the largest speedups (3.77x at 4 threads and 4.34x at 16), showing why speedup alone is not an absolute-performance measure: at 16 threads Basic still takes 1.036 s compared with roughly 0.60-0.68 s for Reduced.

Scheduling strongly affects the Reduced force loop. `Compute_force(part)` performs fewer pair interactions as `part` increases, so default static blocks give early threads more work. At 4 threads, Reduced Default took 1.076 s but Forces Cyclic took 0.676 s; efficiency improved from 54.7% to 85.7%. Using `schedule(static,1)` distributes expensive early and cheaper late iterations across threads more evenly.

Cyclic scheduling on every work-sharing loop was not consistently better. All Cyclic produced the lowest mean runtime in the experiment at 16 threads (0.600 s), only slightly below Forces Cyclic (0.608 s), but Forces Cyclic was faster at 4, 8 and 32 threads. The initialization, reduction and body-update loops have much more uniform iteration costs, so cyclic distribution offers less load-balancing benefit and may reduce locality. The All Cyclic 4-thread result also had high run-to-run variation (sample standard deviation 0.208 s).

Scalability flattened as thread count increased. Basic speedup rose from 3.77x at 4 threads to 4.18x at 8 and 4.34x at 16, then fell to 4.16x at 32. Forces Cyclic rose from 3.43x to 3.64x, 3.81x and 3.75x over the same thread counts. Efficiency fell for every version; Basic dropped from 94.3% at 4 threads to 52.2% at 8, 27.1% at 16 and 13.0% at 32. With only 8 logical CPUs, 16 and 32 threads oversubscribe the machine. Barriers, work-sharing overhead, the Reduced per-thread force arrays and force-reduction work also become more significant as useful work per thread decreases.

Across the tested configurations, Forces Cyclic gives the most consistent Reduced performance, while All Cyclic achieves the single lowest mean at 16 threads. The measurements therefore support applying cyclic scheduling selectively to the imbalanced force loop rather than assuming every loop benefits from cyclic distribution.

7. Conclusion

OpenMP reduced runtime for all four implementations, but performance depended on both algorithm and schedule. The Reduced algorithm was faster because it avoided duplicate pair calculations, and cyclic distribution of its imbalanced force loop improved load balance. Applying cyclic scheduling to every loop gave no consistent extra benefit. Scaling plateaued around the available hardware parallelism and efficiency fell sharply under 16- and 32-thread oversubscription. On this tested system, Reduced Forces Cyclic offered the most consistent overall performance, while Reduced All Cyclic recorded the lowest single mean runtime at 16 threads.